VIRTUAL-DOCTOR COLLABORATION SYSTEM

MINI PROJECT REPORT

Submitted by

AZAD A S

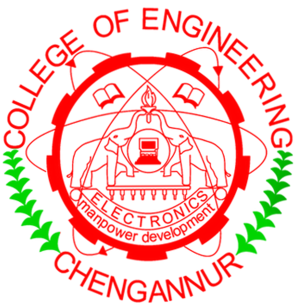
CHN25MCA2021

to

APJ Abdul Kalam Technological University

in partial fulfillment of the requirements for the award of Degree in

Master of Computer Applications



DEPARTMENT OF COMPUTER ENGINEERING

COLLEGE OF ENGINEERING CHENGANNUR, ALAPPUZHA

OCTOBER 2026

DEPARTMENT OF COMPUTER ENGINEERING

COLLEGE OF ENGINEERING CHENGANNUR

ALAPPUZHA

CERTIFICATE

This is to certify that the project report titled ***VIRTUAL-DOCTOR COLLABORATION SYSTEM*** *is a bonafide record of the* ***20MCA245 MINI PROJECT*** *presented by* ***AZAD A S*** *(CHN25MCA2021), Third Semester Master of Computer Applications student, under our guidance and supervision. This project is submitted in partial fulfillment of the requirements for the award of the degree* ***Master of Computer Applications*** *of* ***APJ Abdul Kalam Technological University****.*

Smt. Sruthy R S
Project Guide
Assistant Professor
Dept. of Computer Engineering

Smt. Josmi Jose
Project Coordinator
Assistant Professor
Dept. of Computer Engineering

Dr. Renu George

Professor

Head of the Department

Dept. of Computer Engineering

DECLARATION

I undersigned hereby declare that the project report "**VIRTUAL-DOCTOR COLLABORATION SYSTEM**" , submitted for partial fulfillment of the requirements for the award of degree of Master of Computer Application of the APJ Abdul Kalam Technological University, Kerala is a bonafide work done by me under the supervision of **Smt. Sruthy R S**, Assistant Professor, Department of Computer Engineering. This submission represents my ideas in my own words, and where ideas or words of others have been included, I have adequately and accurately cited and referenced the original sources. I also declare that I have adhered to the ethics of academic honesty and integrity and have not misrepresented or fabricated any data, idea, fact, or source in this submission. I understand that any violation of the above will be a cause for disciplinary action by the institute and/or the University and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been obtained. This report has not been previously formed the basis for the award of any degree, diploma, or similar title of any other University.

Place: Chengannur **AZAD A S**

Date : **CHN25MCA2021**

ACKNOWLEDGEMENT

This work would not have been possible without the support of many people. First and foremost, I give thanks to Almighty God who gave me the inner strength, resources, and ability to complete my project successfully.

I would like to thank **Dr. Hari V.S** , The Principal , College of Engineering Chengannur, for providing the best facilities and atmosphere for the project completion and presentation. Special thanks to **Dr. Renu George** , HOD , Department of Computer Engineering, for her exceptional support, guidance, and encouragement throughout the project. I would also like to thank my project coordinators **Smt. Josmi Jose**, Assistant Professor, Department of Computer Engineering , **Smt. Santhy Viswam** , Assistant Professor, Department of Computer Engineering ,**Smt. Celin T**,Assistant Professor, Department of Computer Engineering , for their extended help and support during the project. I would also like to thank **Smt. Sruthy R S** , Assistant Professor, Department of Computer Engineering, who also took on the role of my project guide , for her extended help and support during the project. I would like to thank my dear friends and faculty for extending their cooperation and encouragement throughout the project work, without which I would never have completed the project this well. Thank you all for your love and also for being very understanding.

AZAD A S

CHN25MCA2021

ABSTRACT

"Virtual-Doctor Collaboration System" is a web-based platform developed to balance the heavy patient crowds at the Outpatient (OP) counters and doctor consultation rooms of government hospitals. Traditional digital systems only shift physical queue tickets into online bookings without increasing doctor capacity, so overcrowding persists because the waiting time of a patient depends directly on the number of available doctors. The proposed MERN-based Smart OPD Load-Balancing System addresses this problem by connecting freelance, retired, and on-leave doctors to temporary shifts at nearby government hospitals. The system provides dynamic doctor onboarding with credential verification, time-bound digital contracts for temporary doctors, and symptom-based triage in which dangerous cases are routed to full-time doctors and normal cases are routed to temporary doctors. Patients can search nearby hospitals, submit their symptoms and history, and track their token in real time, and doctors view a daily patient list in first-come, first-served (FIFO) order and issue digital prescriptions, X-ray requests, and scan requests. The platform supports four types of users, namely the main administrator, the hospital sub-administrator, doctors (fixed and temporary), and patients, each with a separate interface and permitted operations. The system is developed using React.js, HTML5, CSS3, and JavaScript for the frontend, Node.js and Express.js for the backend, MongoDB with the Mongoose ODM as the database, and JSON Web Tokens (JWT) and Multer for authentication and file upload. The system aims to reduce the rush on doctors, balance the daily patient queues, shorten patient waiting times, and support hospital superintendents in making informed capacity and staffing decisions.

CONTENTS

Declaration i

Acknowledgement ii

Abstract iii

List of Figures vii

List of Tables viii

1 INTRODUCTION 1

1.1 Project Area 1

1.2 Objectives 1

2 LITERATURE REVIEW 3

2.1 AI-Enhanced Telemedicine: Transforming Resource Allocation and Cost-Efficiency Analysis via Advanced Queueing Model 3

2.2 Human Resource Shortage in India's Health Sector: A Scoping Review of the Current Landscape 3

2.3 Optimizing Telehealth Utilization During COVID-19: Enhancing Efficiency and Equity Through DEA and ML 3

2.4 Appointment Scheduling Considering Outpatient Unpunctuality Under Telemedicine Services 4

3 PROBLEM DEFINITION AND MOTIVATIONS 5

3.1 Existing System 5

3.2 Limitations 5

3.3 Problem Statement 6

3.4 Proposed System 6

4 REQUIREMENT ANALYSIS 8

4.1 Hardware Requirements 8

4.2 Software Requirements 8

4.3 System Features 9
4.4 Non-Functional Requirements 10
4.4.1 Performance Requirements 10
4.4.2 Quality Requirements 10

5 DESIGN AND IMPLEMENTATION 12
5.1 Overall Design 12
5.2 System Design 12
5.2.1 Client-Side Design 13
5.2.2 Server-Side Design 13
5.3 System Architecture 14
5.4 Use Case Diagram 15
5.5 Data Flow Diagrams (DFD) 16
5.6 Database Design 19
5.7 Module Description 20
5.7.1 User, Hospital and Administration Module 20
5.7.2 Triage, Queue and Consultation Module 21

6 SYSTEM IMPLEMENTATION 22
6.1 Implementation Plan 22
6.1.1 Setup and Environment Preparation 22
6.1.2 Backend Implementation 22
6.1.3 Frontend Implementation 23
6.1.4 Security and Data Handling 23
6.2 Testing and Various Types of Testing Used 24
6.2.1 Unit Testing 24
6.2.2 Integration Testing 25
6.2.3 System Testing 26

7 RESULTS AND DISCUSSION 28
7.1 Advantages 28
7.2 Limitations 29
7.3 Screenshots 30

8 CONCLUSION AND FUTURE SCOPE 34

8.1 Conclusion 34
8.2 Future Scope 34

REFERENCES **35**

LIST OF FIGURES

5.1 System Architecture 15
5.2 Use Case Diagram for Virtual-Doctor Collaboration System 16
5.3 DFD Level 0 17
5.4 DFD Level 1 - Admin 18
5.5 DFD Level 1 - Sub-admin 18
5.6 DFD Level 1 - Doctors 19
5.7 DFD Level 1 - Patients 19

7.1 Home Page 30
7.2 User Registration 30
7.3 User Login 30
7.4 Hospital Admin Dashboard 31
7.5 Live Patient Queue 31
7.6 Temporary Doctor Requirements 31
7.7 Voluntary Doctor Applications and Verifications 32
7.8 Government Hospitals Management 32
7.9 Voluntary Doctor Portal 32
7.10 Doctor Shift and Active Consultation 33

LIST OF TABLES

6.1 Unit Test Cases and Results 25
6.2 Integration Test Cases and Results 26
6.3 System Test Cases and Results 27

CHAPTER 1

INTRODUCTION

1.1 Project Area

Government hospitals struggle to handle heavy patient crowds at the Outpatient (OP) counters and in the doctor consultation rooms. The number of patients arriving at a hospital on a given day is large, while the number of doctors available to attend to them is limited. As a result, patients face long waiting times, and doctors face a heavy rush during the working hours of the OPD.

Conventionally, digital systems for hospitals only shift the physical queue ticket into an online booking. Such systems make it easier to obtain a token, but they do not increase the number of doctors who can attend to the patients. Overcrowding therefore persists, because the waiting time of a patient depends directly on the total number of available doctors. At the same time, many freelance, retired, and on-leave doctors are willing to work for short periods, but there is no organized way to connect them to the hospitals that need them.

" **Virtual-Doctor Collaboration System** " is proposed as a MERN-based Smart OPD Load-Balancing System that connects freelance, retired, and on-leave doctors to temporary shifts at nearby government hospitals. It provides dynamic doctor onboarding with credential verification, time-bound digital contracts, symptom-based triage, a First-In, First-Out (FIFO) patient queue with token numbers, and a digital clinical workspace for doctors. The platform supports four types of users, namely the main administrator, the hospital sub-administrator, doctors (fixed and temporary), and patients, each with a separate interface and permitted operations.

Developed using React.js, Node.js, Express.js, MongoDB, and web technologies, the system combines web application development, database management, and queue-balancing logic in a practical system for everyday use. By using the web application, government hospitals can reduce the hospital rush and balance the patient queues efficiently.

1.2 Objectives

1. **To develop an integrated web platform for balancing OPD crowds:** The system aims to provide a single web platform where the main administrator, hospital sub-administrators, doctors, and patients can carry out their activities related to OPD management. The platform

reduces the dependence on the fixed number of full-time doctors of a hospital.

2. **To onboard temporary doctors for flexible shifts:** The system connects retired, freelance, and off-duty doctors to short-term shifts at nearby government hospitals, with the verification of their medical proofs and credentials before the assignment of shifts.
3. **To enforce time-bound digital contracts:** The system generates and enforces binding time-based digital agreements for temporary doctors, so that their shift commitments are secured before duty hours are assigned.
4. **To triage patients based on symptom severity:** The system collects the symptoms and history of patients, checks how serious the illness is, and routes dangerous cases to full-time doctors and normal cases to temporary doctors.
5. **To reduce patient waiting times through an ordered queue:** The system organizes patients into a first-come, first-served queue with token numbers, shows doctors their daily patient list in that order, and allows patients to track their token position in real time.
6. **To provide a digital clinical workspace for doctors:** The system enables doctors to view the patient list, search patient records, and issue digital prescriptions and X-ray and scan requests.
7. **To support hospital management with dashboards and analytics:** The system helps hospital superintendents balance daily crowds by displaying hospital overcrowding trends, doctor shift allocations, and patient visit logs.
8. **To provide secure role-based access:** The system provides registration and login using JSON Web Tokens with role-based permissions so that the main administrator, sub-administrators, doctors, and patients can access only the operations permitted for their roles.

CHAPTER 2

LITERATURE REVIEW

2.1 AI-Enhanced Telemedicine: Transforming Resource Allocation and Cost-Efficiency Analysis via Advanced Queueing Model

This paper (2025) uses artificial intelligence together with queueing theory, including Q-Learning, to manage patient queues and the allocation of doctors. The study reports that the waiting time of patients decreased by about 40% and that the resources of the hospital were used more efficiently. This paper is relevant to the Virtual-Doctor Collaboration System because it shows that the organized management of patient queues and doctor allocation reduces waiting time, which is the main aim of the proposed queue-balancing logic. However, the approach was tested mainly in one hospital, so the results may differ elsewhere, and it does not provide a workflow for adding more doctors at a hospital. [1]

2.2 Human Resource Shortage in India's Health Sector: A Scoping Review of the Current Landscape

This paper (2024) is a scoping review of 88 research studies about the shortage of healthcare workers in India. It finds that the shortages are caused by a lack of training, the migration of trained staff, job dissatisfaction, and poor recruitment, and that rural areas were affected the most. This paper is relevant to the proposed system because it establishes that the shortage of doctors is a real and serious bottleneck in the Indian health sector, which motivates the onboarding of temporary doctors. However, the work only analyzes the problem and does not provide a practical software solution. [2]

2.3 Optimizing Telehealth Utilization During COVID-19: Enhancing Efficiency and Equity Through DEA and ML

This paper (2025) uses Data Envelopment Analysis (DEA) and machine learning to study the efficiency of telehealth in counties of the United States. It finds that rural areas had lower telehealth

efficiency because of poor internet access and workforce shortages. This paper is relevant to the proposed system because it shows that workforce shortages directly reduce the efficiency of healthcare delivery. However, the study is based on COVID-19 data from the United States, so its findings may not apply everywhere. [3]

2.4 Appointment Scheduling Considering Outpatient Unpunctuality Under Telemedicine Services

This paper (2025) creates a scheduling model that considers late arrivals, telemedicine patients, and the workload of doctors. The model reduced the overtime of doctors and the switching between online and offline consultations. This paper is relevant to the proposed system because it also considers the workload of doctors and the order of patients when scheduling consultations. However, it focuses mainly on appointment scheduling and not on increasing the number of available doctors.

Related work on integrated resource scheduling for patient appointment series, which designs tactical blueprints for an orthopedic clinic [4], also shows the importance of planning the scheduling of limited clinical resources.

CHAPTER 3

PROBLEM DEFINITION AND MOTIVATIONS

3.1 Existing System

In existing hospital systems, the patient crowd at the OP counters and in the consultation rooms is handled mostly through physical queue tickets or through online booking systems that only move the same ticket into an online form. These systems do not increase the number of doctors, so the waiting time of a patient still depends directly on the number of doctors available. Hospitals have no organized way to bring retired, freelance, or on-leave doctors to work short-term shifts during peak periods, and patients are not triaged by the seriousness of their illness. Existing research addresses appointment scheduling, queue management, or telehealth efficiency as individual problems, and most of it does not solve the doctor shortage bottleneck or provide a complete workflow for administrators, doctors, and patients. The proposed Virtual-Doctor Collaboration System addresses these limitations by providing a single web platform that connects the main administrator, hospital sub-administrators, doctors, and patients and combines dynamic doctor onboarding, digital contracts, symptom-based triage, and a FIFO patient queue.

3.2 Limitations

- **Doctor Shortage Bottleneck:** Most existing systems focus on basic online OP ticket booking rather than on the shortage of doctors.
- **Long Patient Waiting Times:** The waiting time depends directly on the number of available doctors, so heavy crowds lead to long waiting.
- **Heavy Rush on Doctors:** Full-time doctors have to attend to all patients, including routine cases, during peak hours.
- **No Dynamic Onboarding of Temporary Doctors:** There is limited use of retired, freelance, or on-leave doctors to handle peak hospital crowds.
- **No Symptom-Based Triage:** Patients are not classified by symptom severity, so dangerous and routine cases wait in the same queue.
- **Lack of Time-Bound Digital Contracts:** There is little focus on combining doctor availabil-

ity types with binding time-based digital agreements.

- **Manual Verification of Credentials:** The medical proofs of temporary doctors are difficult to verify and track before shifts are assigned.
- **Isolated Research Solutions:** Existing work treats appointment scheduling, queue management, and telehealth efficiency as separate problems.
- **Lack of Integrated Operational Platform:** Existing approaches do not provide a simple, practical, and low-cost workflow connecting administrators, doctors, and patients.
- **Limited Operational Reporting:** Hospital superintendents lack dashboards for overcrowding trends, doctor shift allocations, and patient visits.

3.3 Problem Statement

To develop a web-based system that balances government hospital OPD crowds by onboarding temporary doctors for flexible shifts and triaging patients based on symptom severity, reducing the doctor-side rush and the waiting times of patients.

3.4 Proposed System

The Virtual-Doctor Collaboration System is a single MERN-based web platform for the main administrator, hospital sub-administrators, doctors, and patients. It signs up retired, freelance, or off-duty doctors to work short-term shifts at nearby government hospitals. The system checks the schedules, duty hours, medical proofs, and signed agreements of doctors before assigning shifts. Patients search nearby hospitals and submit their symptoms and history, and the system checks the symptoms to see how serious the illness is. Dangerous cases are sent to full-time doctors and normal cases are sent to temporary doctors. Doctors see their daily patient list in first-come, first-served order and issue digital prescriptions, X-ray requests, and scan requests. The system is built using React.js, Node.js with Express.js, and MongoDB, and it helps hospital managers balance the daily crowds and reduce the waiting times of patients. The key features of the proposed system include:

1. **User Registration and Login :** Allows users to register and securely log in using JSON Web Tokens, with role-based access for the main administrator, sub-administrators, doctors, and patients.
2. **Hospital Onboarding :** Allows the main administrator to register, modify, remove, and locate government hospitals and to assign hospital sub-administrators.
3. **Fixed Doctor Management :** Allows hospital sub-administrators to add fixed doctors with

their working hours, consulting dates, and duty schedules.

4. **Temporary Doctor Signup and Verification :** Allows freelance, retired, and off-duty doctors to register, upload credential proofs, and apply for shifts, and allows sub-administrators to verify the proofs.
5. **Digital Contract Management :** Generates time-bound digital contracts that temporary doctors sign before their shifts.
6. **Symptom Submission and Triage :** Collects patient symptoms and history and routes cases to fixed or temporary doctors based on severity.
7. **Priority FIFO Queue and Token Tracking :** Orders patients in first-come, first-served order with token numbers and allows patients to track their token in real time.
8. **Digital Clinical Workspace :** Allows doctors to view the patient list, search patient records, and issue digital prescriptions and X-ray and scan requests.
9. **Administration and Analytics :** Allows the main administrator to view all platform records and monitor analytics, and displays overcrowding trends, doctor shift allocations, and patient visit logs.
10. **Notifications :** Informs users about appointment, contract, and queue updates.

CHAPTER 4

REQUIREMENT ANALYSIS

4.1 Hardware Requirements

- **Computer/Laptop:** Minimum 4 GB RAM and a dual-core central processor
- **Storage Space:** At least 10 GB of available disk capacity
- **Connectivity:** Active network connection for web application access
- **Visual Output:** Monitor or display unit for viewing system interfaces
- **Optional Device:** Smartphone or tablet for accessing portal screens via web browsers

4.2 Software Requirements

- **Operating System:** Windows, Linux, or macOS
- **Programming Languages:**
  - **JavaScript (ES6+) and Node.js:** Used for backend development
  - **HTML5, CSS3, JavaScript:** Used for frontend development
- **Framework:**
  - **React.js:** Used to build the interactive frontend dashboards and input forms
  - **Node.js with Express.js:** Used to build the backend web application and handle routing, business logic, and REST API endpoints
- **Database:**
  - **MongoDB with Mongoose ODM:** Used to store users, hospitals, schedules, contracts, patient visits, and consultations.
- **Libraries:**
  - **JSON Web Tokens (JWT):** Used for user authentication.
  - **Multer:** Used for the upload of credential proofs and other files.
  - **JavaScript Data Structures and Utility Functions:** Used for data processing, such as

the FIFO queue ordering.

- **Development Tools:**
  - **VS Code:** For coding and development
  - **Web Browser (Google Chrome, Mozilla Firefox, or Microsoft Edge):** For running and testing the application

4.3 System Features

1. **Facility Onboarding:** Allows the main administrator to register, modify, remove, and locate government healthcare centers.
2. **Staff Roster Entry:** Allows hospital sub-administrators to input the details of fixed doctors with their working hours, consulting dates, and duty schedules.
3. **Temporary Doctor Signup:** Allows freelance, retired, and off-duty doctors to register with document verification and to send shift requests.
4. **Digital Contract Management:** Generates and enforces binding time-based agreements for the shifts of temporary doctors.
5. **Symptom Submission and Triage:** Collects patient health information and routes cases based on the severity level.
6. **FIFO Queue Engine:** Organizes the patient consultation sequence and assigns priority token numbers.
7. **Digital Clinical Workspace:** Enables doctors to view patient lists, search records, and prescribe medicines, X-rays, or scans.
8. **Global Platform Oversight:** Provides secure main administrative access to monitor cross-hospital records and system activity.
9. **User Registration and Login:** Allows users to register and authenticates the main administrator, sub-administrators, doctors, and patients with JSON Web Tokens and directs them to the interface for their role.
10. **Hospital Search:** Allows patients to search nearby hospitals and view their capacity and clinical departments.
11. **Dashboards and Reports:** Provides dashboards for overcrowding trends, doctor shift allocations, and patient visit logs.

12. **Logout:** Allows users to securely end their sessions.
13. **Responsive Web Interface:** Provides a simple interface that adapts to desktop, tablet, and mobile browser sizes.

4.4 Non-Functional Requirements

4.4.1 Performance Requirements

1. **Speed:** The web application must render dashboards and process patient token requests rapidly.
2. **Fast Queue Updates:** Token and queue information should be returned without unnecessary delay.
3. **Efficient Database Access:** The system should retrieve and store data efficiently in the MongoDB database to support real-time token sequencing and administrative reports.
4. **Report Generation:** Dashboards and reports should be generated within a reasonable time.
5. **Smooth Interaction:** The interface should allow smooth navigation between the hospital, queue, consultation, and dashboard pages.

4.4.2 Quality Requirements

1. **Usability:** The interface must remain clean, clear, and simple for hospital staff, doctors, and patients to operate.
2. **Reliability:** The web service must deliver stable, error-free queue management and accurate shift schedules.
3. **Security:** Medical credentials, contract documents, and patient records must be safely encrypted and restricted.
4. **Maintainability:** Separate modules for hospital and doctor management, doctor registration and contracts, triage and queue, consultation, and analytics should make the system easy to maintain.
5. **Scalability:** The system must easily handle an increasing number of registered hospitals, doctors, and patient visits.
6. **Responsiveness:** The system layouts must automatically adapt to desktop screens, tablets, and mobile web browsers.
7. **Compatibility:** The application should work properly on commonly used modern web browsers.

8. **Data Integrity:** The system should maintain accurate and consistent hospital, schedule, contract, and visit data in the database.

CHAPTER 5

DESIGN AND IMPLEMENTATION

The proposed system is a web-based application named Virtual-Doctor Collaboration System, designed to help government hospitals balance their OPD crowds. The system provides a centralized platform where patients can search hospitals and submit symptoms, temporary doctors can apply for shifts and sign contracts, fixed doctors can manage their schedules, hospital sub-administrators can manage their hospitals, and the main administrator can monitor the whole platform. The application runs on a Node.js and Express.js backend with a MongoDB database and provides JSON endpoints through a REST API.

5.1 Overall Design

The Virtual-Doctor Collaboration System is a web-based system designed to combine dynamic doctor onboarding, digital contracts, symptom-based triage, and FIFO queue management. It follows a client-server design that uses React.js, HTML, CSS, and JavaScript for the user interface, Node.js and Express.js for server-side processing, and MongoDB for data storage. The system allows patients to search hospitals, submit symptoms, and track their token, allows doctors to apply for shifts, sign contracts, and consult patients, and allows administrators to manage hospitals, doctors, and reports. Overall, the system aims to provide a simple and organized platform for balancing patient queues in government hospitals.

5.2 System Design

The system is structured into several layers: user layer, frontend layer, backend layer, database layer, the load-balancing and routing engine, and the analytics and administrative tools. The user layer provides separate web interfaces for the main administrator, the hospital sub-administrator, doctors (fixed and temporary), and patients. The frontend layer is built with React.js, HTML, CSS, and JavaScript and delivers interactive dashboards and input forms that display patient queues, schedules, and practitioner files. The backend layer uses Node.js and Express.js to execute server operations and the core application workflows, and a REST API layer provides JSON endpoints over secure HTTP communication. The business logic and services of the backend include user authentication with JWT, the hospital roster engine, the patient queue (FIFO), contract verification,

symptom data handling, and the consultation manager. The database layer uses MongoDB as the primary database, and the load-balancing and routing engine evaluates the symptom severity of the patients, routes severe cases to permanent staff and routine cases to temporary doctors, and produces ordered FIFO queue tokens. The analytics and administrative tools display the daily patient traffic, duty rosters, and hospital metrics. When a user interacts with the system, the request is sent from the React frontend to the Express API, processed by the FIFO queue logic, stored in the MongoDB database, and the response is shown in the real-time dashboards. This layered design ensures organized processing, maintainability, and consistent management of hospital, contract, and queue data.

5.2.1 Client-Side Design

The client-side represents the user interface of the system. It is developed using React.js, HTML, CSS, and JavaScript to provide a simple, responsive, and user-friendly experience.

The client-side includes pages such as the home page, the hospital directory and capacity finder, registration, login, the main administrator dashboard, hospital and user management pages, the credential verification page for doctors, the patient symptom submission and token tracking pages, and the doctor workspace. HTML structures the pages, CSS provides the design and responsiveness, and React.js and JavaScript are used for the interactive dashboards and form handling.

Users can access the system through a web browser and perform actions such as searching nearby hospitals, submitting symptoms and history, monitoring the queue position, uploading credential proofs, applying for shifts, signing contracts, and issuing digital prescriptions. The dashboards summarize hospitals, shift contracts, doctor verifications, and patient visits.

5.2.2 Server-Side Design

The server-side is responsible for handling the application logic, processing user requests, and communicating with the database. It is developed using Node.js and Express.js with the Mongoose ODM.

The server processes operations such as authentication and role control with JWT, hospital and doctor management, verification of credential proofs, generation and verification of contracts, symptom data handling, triage and routing, FIFO queue management, consultation handling, and report preparation. It communicates with the MongoDB database to store and retrieve users, hospitals, schedules, contracts, patient visits, and consultations.

For triage, the server evaluates the symptom severity submitted by the patient, routes danger-

ous cases to full-time doctors and normal cases to temporary doctors, and assigns the next FIFO token. For temporary doctors, the server checks the schedule, duty hours, medical proof, and signed agreement before a shift is assigned. When a user performs an action, the request is sent to the Express server, validated, processed, and the appropriate response is returned to the client-side.

5.3 System Architecture

The system follows a layered architecture consisting of the User Layers (React interfaces), the Backend Layer (Node.js and Express), and the Database Layer (MongoDB).

The user layers consist of the Main Admin, the Hospital Sub-Admin, Doctors (fixed and temporary), and Patients. The Main Admin uses a web dashboard to manage hospitals, view all platform records, and monitor analytics. The Hospital Sub-Admin uses a hospital portal to register the hospital location, add fixed doctors, verify the proofs of temporary doctors, and issue digital contracts. Doctors use a doctor workspace to view the priority FIFO queue list, issue digital prescriptions, set shifts (fixed doctors), and apply for and sign contracts (temporary doctors). Patients use a patient app to search nearby hospitals, submit symptoms and history, and track their token in real time. The backend layer provides the business logic and services, namely user authentication (JWT), the hospital roster engine, the patient queue (FIFO), contract verification, symptom data handling, and the consultation manager, together with a REST API layer that carries JSON endpoints over secure HTTP communication. The database layer stores the document collections users, hospitals, schedules, contracts, patient_visits, and consultations.

The core control components of the system are the Load-Balancing and Routing Engine, which evaluates the severity of submitted symptoms, routes severe cases to permanent staff and routine cases to temporary doctors, and produces ordered FIFO queue tokens; the Hospital Roster Engine, which checks doctor schedules and duty hours; and the Contract Verification service, which validates medical proofs and signed agreements before shifts are assigned. Authentication with JWT and role-based permissions separate the actions of the main administrator, sub-administrators, doctors, and patients. The data flow of the system is: User Action, React Frontend, Express API, FIFO Queue Logic, MongoDB Update, and Real-Time Dashboards.

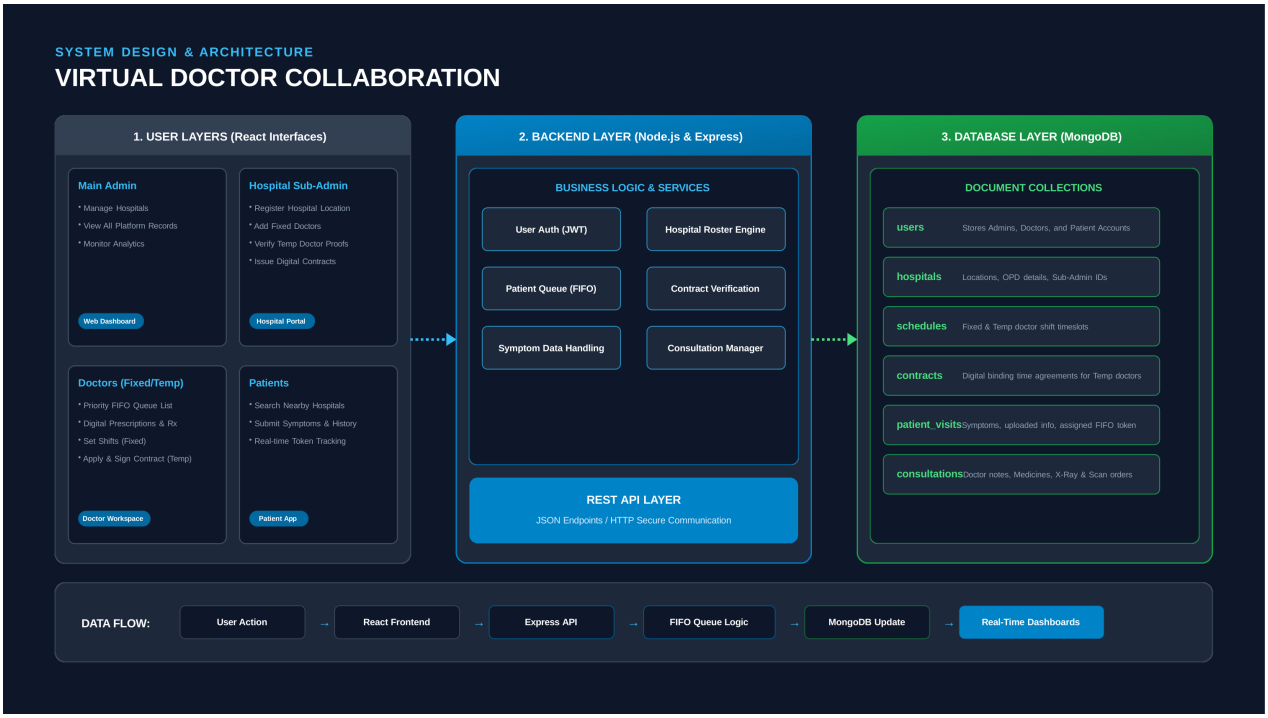


Figure 5.1: System Architecture

5.4 Use Case Diagram

The Use Case Diagram represents the interaction between the users and the Virtual-Doctor Collaboration System. It identifies the main users of the system and the activities that can be performed. In the diagram, there are four actors: **Admin**, **Sub-admin**, **Doctor**, and **Patient**.

Actors:

1. **Admin:** The main administrator can log in, administer hospitals, manage users, and verify users.
2. **Sub-admin:** The hospital sub-administrator can register and log in, post doctor jobs, and manage appointments for the hospital.
3. **Doctor:** The doctor, either fixed or temporary, can register and log in and can view and respond to the doctor jobs posted by hospitals.
4. **Patient:** The patient can register and log in, book an appointment, and manage the appointment and queue status.

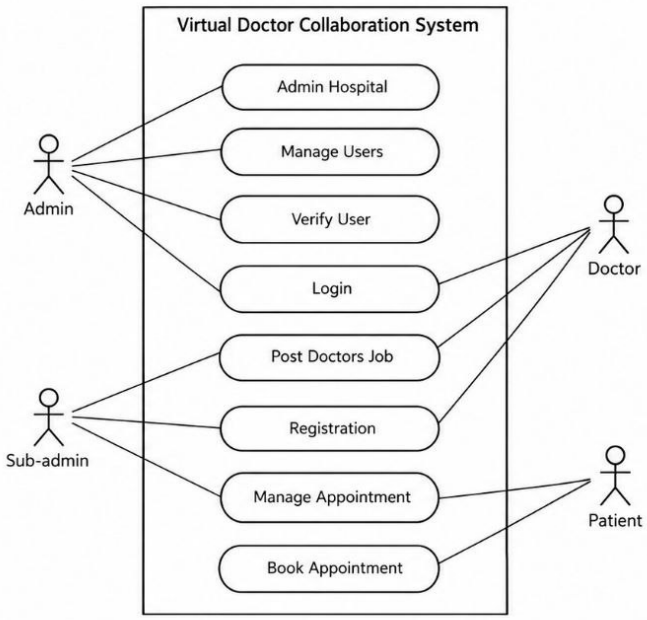


Figure 5.2: Use Case Diagram for Virtual-Doctor Collaboration System

5.5 Data Flow Diagrams (DFD)

The Data Flow Diagram (DFD) illustrates how data flows within the system between the users, the application processes, and the data stores. The system has four external entities, namely the Admin, the Sub-admin, the Doctors, and the Patients, and the data stores User-DB, Hospital-DB, Doctor-DB, Doctors-DB, Patients-DB, appointment-DB, Availability-DB, Profile-DB, and Document-info.

The Level 0 DFD represents the whole system as a single process, the Virtual-Doctor Collaboration System, interacting with the external users and the database. The Level 1 DFDs divide this process into sub-processes for each of the four users and are shown separately below.

The Level 1 DFD of the **Admin** contains the processes Login (1.0), View All Medical (1.1), Manage Hospital (1.2), Manage Doctors (1.3), and View Patient Info (1.4). The admin logs in and the user information is checked against the User-DB, the hospital records can be viewed, updated, or deleted in the Hospital-DB, the doctor records can be viewed or deleted in the Doctor-DB, and the patient information is viewed from the User-DB.

The Level 1 DFD of the **Sub-admin** contains the processes Register (1.0), Login (1.1), Register and Update Hospital (1.2), Manage Doctor (1.3), and Register Temp Doctor (1.4). The sub-admin registers and logs in, the credentials are validated, the hospital details are stored and updated in the Hospital-DB, the fixed doctor details are managed in the Doctor-DB, and the details of temporary doctors are registered in the User-DB and the Doctor-DB.

The Level 1 DFD of the **Doctors** contains the processes Login (1.0), Manage Profile (1.5), Upload Medical Document (1.6), Schedule (1.7), Search Patients (1.8), and Consultation (1.9). The doctor logs in, manages the profile in the Profile-DB, edits the availability that is stored in the Availability-DB, searches the patient information from the Patients-DB and the appointment-DB, views the appointments, and records the consultation notes and diagnosis information in the Document-info store.

The Level 1 DFD of the **Patients** contains the processes Login (1.0), Register (1.2), Search Doctors (1.3), Book Appointment (1.4), Manage Profile (1.5), and Upload Medical Document (1.6). The patient registers and logs in, searches the doctor information from the Doctors-DB, books an appointment that is stored in the appointment-DB, manages the profile in the Profile-DB, and uploads medical documents that are stored in the Document-info store.

Overall, the DFD provides a clear understanding of how the different processes of the system interact and how data is processed, stored, and retrieved.

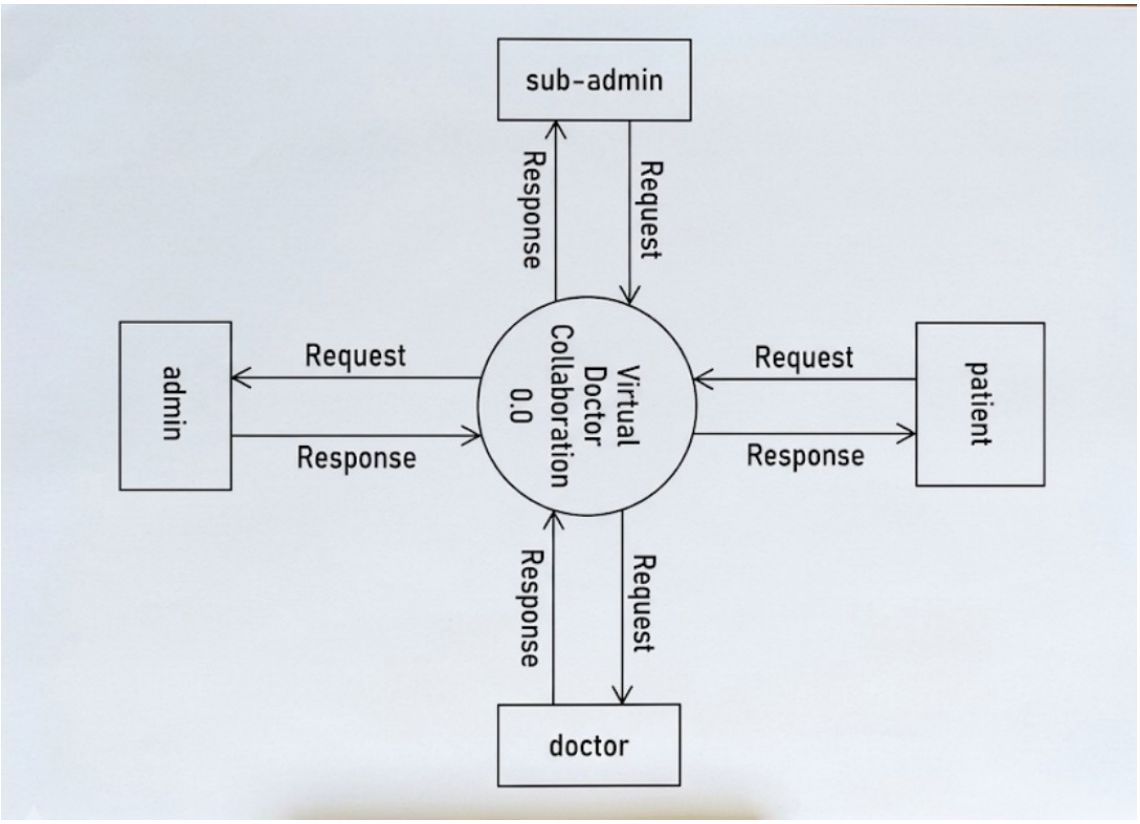


Figure 5.3: DFD Level 0

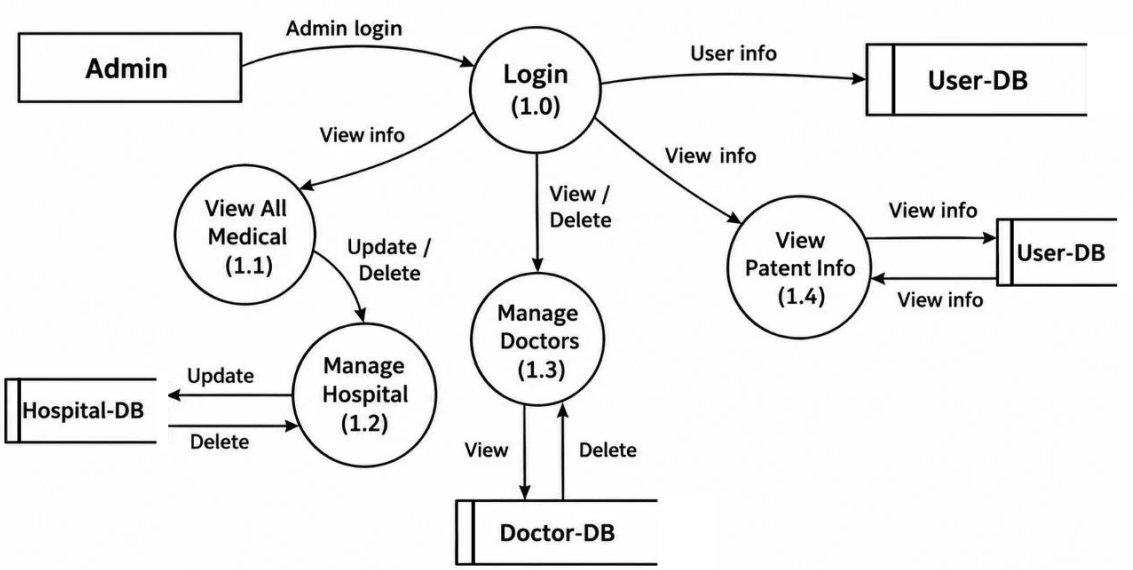


Figure 5.4: DFD Level 1 - Admin

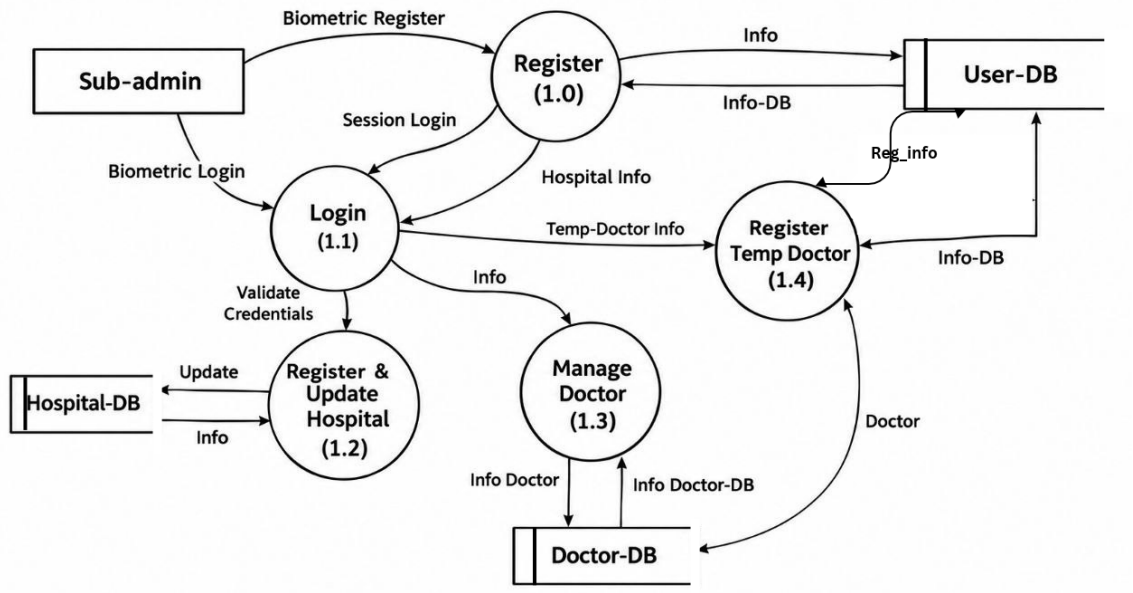


Figure 5.5: DFD Level 1 - Sub-admin

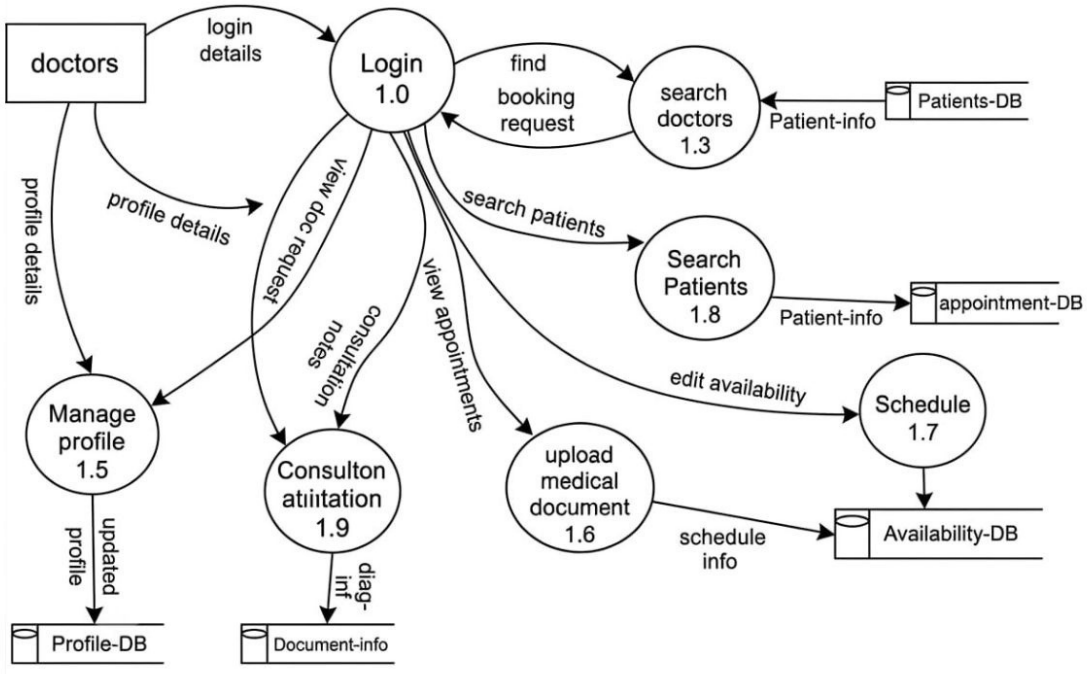


Figure 5.6: DFD Level 1 - Doctors

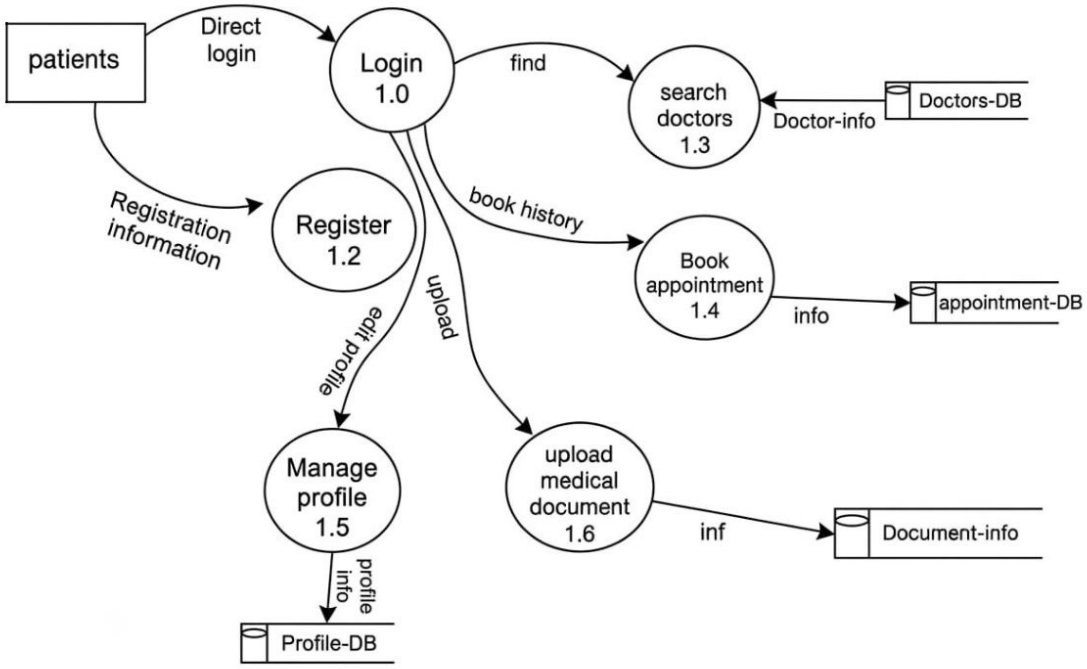


Figure 5.7: DFD Level 1 - Patients

5.6 Database Design

The system uses MongoDB, a NoSQL document database, as its data store, with the Mongoose ODM used to define the schemas. The database shall provide protected, structured, and scalable

information management, and it connects related schemas between hospital entities, doctor duty rosters, and patient appointments. It supports adding, updating, deleting, and fetching of document records, and it ensures fast record retrieval to support real-time token sequencing and administrative reports. The database stores the following information, and access to the data is kept secure and consistent through validation and role-based access.

- **users:** Accounts of the administrators, doctors, and patients
- **hospitals:** Hospital locations, OPD details, and sub-administrator IDs
- **schedules:** Shift timeslots of fixed and temporary doctors
- **contracts:** Digital binding time agreements for temporary doctors
- **patient_visits:** Symptoms, uploaded information, and the assigned FIFO token
- **consultations:** Doctor notes, medicines, and X-ray and scan orders

5.7 Module Description

Virtual-Doctor Collaboration System is designed as a role-based web application that enables the main administrator, hospital sub-administrators, doctors, and patients to balance the OPD crowd of government hospitals. It operates through two primary modules **User, Hospital and Administration Module and Triage, Queue and Consultation Module** which work together to provide an organized platform for balancing the OPD load.

5.7.1 User, Hospital and Administration Module

This module covers the user interface, the hospital and doctor management, the registration of doctors with contracts, the database, and the analytics and reporting functions.

1. **Frontend UI:** Provides an intuitive and adaptive user portal for all platform roles and displays medical facilities, work rosters, patient queues, and digital consultation records.
2. **Hospital and Doctor Management Module:** Adds, updates, searches, and verifies government hospitals and permanent staff, and manages clinical profiles and practitioner credentials.
3. **Doctor Registration and Contract Module:** Onboards freelance, retired, and off-duty physicians with credential verification and generates time-bound digital agreements to secure temporary shift commitments.
4. **Database Module:** Uses MongoDB to store user profiles, hospital locations, shift schedules, dynamic contracts, and medical logs.

5. **Analytics and Administrative Module:** Displays hospital overcrowding trends, doctor shift allocations, and patient visit logs, and helps hospital superintendents make informed capacity and staffing decisions.

5.7.2 Triage, Queue and Consultation Module

This module is the scheduling component that controls the symptom triage, the patient queue, and the consultations.

1. **Symptom Triage and Queue Module:** Categorizes the health conditions of patients based on symptom severity, routes cases to fixed or temporary doctors, and maintains First-In, First-Out token ordering.
2. **Consultation Workspace Module:** Displays prioritized patient queues for attending medical practitioners and enables searching of patient records and issuing of digital prescriptions, X-rays, and diagnostic scans.
3. **Load-Balancing and Routing Engine:** Evaluates the submitted symptom severity, routes severe cases to permanent staff and routine cases to temporary doctors, and produces the ordered FIFO queue tokens.
4. **Token Tracking:** Allows patients to monitor their live queue position in real time.

CHAPTER 6

SYSTEM IMPLEMENTATION

6.1 Implementation Plan

6.1.1 Setup and Environment Preparation

- Set up the development environment using Node.js, npm, and VS Code.
- Install the required libraries for React.js, Express.js, Mongoose, JSON Web Tokens (JWT), and Multer.
- Configure the MongoDB database to store user, hospital, schedule, contract, patient visit, and consultation information.
- Create the necessary document schemas using Mongoose for users, hospitals, schedules, contracts, patient visits, and consultations.
- Organize the project into modules for hospital and doctor management, doctor registration and contracts, symptom triage and queue, consultation workspace, and analytics and administration.
- Set up the React frontend and connect it to the Express REST API.

6.1.2 Backend Implementation

- Develop the backend using Node.js and Express.js to handle routing, business logic, and application workflows.
- Implement user registration, login, and logout with JWT-based authentication and role-based permissions for the main administrator, sub-administrators, doctors, and patients.
- Provide JSON endpoints through the REST API layer.
- Implement the Hospital Roster Engine for fixed and temporary doctor shift schedules.
- Implement the Contract Verification service for medical proofs and digital contracts, with file uploads handled by Multer.
- Implement the symptom triage logic that routes severe cases to full-time doctors and routine

cases to temporary doctors.

- Implement the Patient Queue (FIFO) logic that assigns the ordered token numbers.
- Implement the Consultation Manager for doctor notes, medicines, and X-ray and scan orders.
- Implement the analytics data for overcrowding trends, doctor shift allocations, and patient visits.

6.1.3 Frontend Implementation

- Develop the user interface using React.js, HTML5, CSS3, and JavaScript.
- Design pages for the home page, registration, login, hospital directory and capacity finder, patient symptom submission, token tracking, and the doctor workspace.
- Design dashboards for the main administrator and the hospital sub-administrators, and pages for hospital management, user governance, and credential verification.
- Create forms with validation to prevent invalid or incomplete submissions.
- Display the priority FIFO queue list to doctors and the real-time token position to patients.
- Use React components to display user information, hospitals, queue status, and stored records dynamically.
- Design the interface to be responsive and suitable for desktop, tablet, and mobile browser sizes.

6.1.4 Security and Data Handling

- Implement JWT authentication to protect user accounts and operational functions.
- Implement role-based permissions to separate the actions of the main administrator, sub-administrators, doctors, and patients.
- Verify the medical credentials and signed contracts of temporary doctors before shifts are assigned.
- Validate login credentials before granting access to protected pages.
- Validate form inputs and uploaded files before processing.
- Encrypt and restrict medical credentials, contract documents, and patient records.
- Store user, hospital, schedule, contract, visit, and consultation information in the MongoDB database.

- Ensure that protected operations are accessible only to authenticated users with the required role.

6.2 Testing and Various Types of Testing Used

Testing verifies that the software functions correctly according to its requirements, helping identify errors and ensure reliable interaction between the frontend, backend, database, and queue-balancing logic. In the Virtual-Doctor Collaboration System, testing covers user registration and login, hospital management, doctor onboarding, contract handling, symptom triage, FIFO queue management, consultation handling, database accuracy, input validation, and access control.

6.2.1 Unit Testing

Unit testing checks individual functionalities independently to verify that each performs its intended task. In this system, individual features such as registration, login, hospital creation, doctor registration, contract generation, symptom triage, and FIFO token assignment are tested separately. This helps identify errors in individual components before testing the complete application.

Test Case	Description	Expected Result	Actual Result	Pass/Fail
1	User Registration	A new user account is created with valid details.	The account was created and stored in the database.	Pass
2	User Login	A registered user logs in using valid credentials and is directed to the interface for the role.	The user logged in and was directed to the correct role interface.	Pass
3	Invalid Login	Incorrect login credentials are rejected with an error message.	The login was rejected and an error message was displayed.	Pass
4	Add Hospital	A hospital is saved with its location and OPD details.	The hospital was saved with its location and OPD details.	Pass
5	Temporary Doctor Signup	A temporary doctor is registered with the uploaded credential proof.	The doctor was registered and the credential proof was uploaded.	Pass
6	Digital Contract Generation	A time-bound digital contract is generated for a verified temporary doctor.	The time-bound contract was generated for the verified doctor.	Pass
7	Symptom Triage	Severe cases are routed to full-time doctors and routine cases to temporary doctors.	Severe cases went to full-time doctors and routine cases to temporary doctors.	Pass
8	FIFO Token Assignment	Patients are assigned token numbers in first-come, first-served order.	Token numbers were assigned in first-come, first-served order.	Pass

Table 6.1: Unit Test Cases and Results

6.2.2 Integration Testing

Integration testing verifies that different components of the system work together correctly. It checks the interaction between the React frontend, the Express backend, the MongoDB database, and the file upload services. The testing focuses on whether user actions are processed correctly, database changes are reflected in the interface, and queue and token results are returned and displayed as expected.

Test Case	Description	Expected Result	Actual Result	Pass/Fail
1	Registration and Database Integration	Valid user details are stored in the database.	The user details were stored in the database.	Pass
2	Login Integration	The backend verifies credentials and issues a JWT with the appropriate role.	The backend verified the credentials and issued a JWT with the correct role.	Pass
3	Credential Upload Integration	The proofs uploaded by a temporary doctor are stored and shown to the sub-admin for verification.	The uploaded proofs were stored and shown to the sub-admin.	Pass
4	Contract Integration	A signed contract is stored and the shift of the doctor appears in the schedule.	The signed contract was stored and the shift appeared in the schedule.	Pass
5	Symptom and Queue Integration	A patient who submits symptoms receives a token and appears in the queue of the assigned doctor.	The patient received a token and appeared in the doctor's queue.	Pass
6	Consultation Integration	A prescription or scan request issued by the doctor is stored and shown for the patient.	The prescription or scan request was stored and shown for the patient.	Pass
7	Hospital Search Integration	Hospitals and their details are displayed correctly in the hospital directory.	The hospitals and their details were displayed correctly.	Pass
8	Analytics Integration	Stored visit and shift data are retrieved correctly to display the dashboards.	The visit and shift data were retrieved and the dashboards displayed correctly.	Pass

Table 6.2: Integration Test Cases and Results

6.2.3 System Testing

System testing evaluates the complete application to verify that its major features operate together as expected. It includes testing user access, hospital search, doctor onboarding, triage, queue management, consultation, administration, and database operations. It also checks input validation and access restrictions to identify problems that may affect the overall reliability and usability of the application.

Test Case	Description	Expected Result	Actual Result	Pass/Fail
1	Overall System Testing	The major features of the application operate correctly.	All major features of the application operated correctly.	Pass
2	Authentication Testing	Valid credentials are accepted and invalid credentials are rejected.	Valid credentials were accepted and invalid credentials were rejected.	Pass
3	Hospital Search Testing	Hospitals can be searched and filtered by district and clinical department.	Hospitals were searched and filtered by district and clinical department.	Pass
4	Doctor Onboarding Testing	Temporary doctors can register, be verified, and sign contracts before shifts are assigned.	Temporary doctors registered, were verified, and signed contracts before shifts were assigned.	Pass
5	Triage and Queue Testing	Patients are triaged by symptom severity and the queue status is displayed correctly in FIFO order.	Patients were triaged by severity and the queue was displayed in FIFO order.	Pass
6	Administrator Testing	The main administrator and sub-administrators can perform only their permitted management operations.	Each administrator could perform only the permitted operations.	Pass
7	Input Validation Testing	Invalid or incomplete input is rejected appropriately.	Invalid or incomplete input was rejected with an appropriate message.	Pass
8	Access Control Testing	Users cannot access pages or records that are not permitted for their roles.	Access to pages and records not permitted for the role was denied.	Pass

Table 6.3: System Test Cases and Results

CHAPTER 7

RESULTS AND DISCUSSION

7.1 Advantages

- **Integrated Platform:** The system combines hospital discovery, doctor onboarding, digital contracts, symptom triage, queue management, and consultation in one application.
- **Expanded Clinical Capacity:** Freelance, retired, and off-duty doctors are connected to temporary shifts using digital contracts, which expands the clinical capacity of government hospitals dynamically.
- **Symptom-Based Triage:** Dangerous cases are sent to full-time doctors and normal cases to temporary doctors, which optimizes the distribution of the queue and shortens the waiting delays.
- **Verified Temporary Doctors:** Doctor schedules, duty hours, medical proofs, and signed agreements are checked before shifts are assigned.
- **FIFO Queue and Token Tracking:** Doctors see their daily patient list in first-come, first-served order, and patients can track their token position in real time.
- **Role-Based Workflows:** Separate interfaces and permissions for the main administrator, sub-administrators, doctors, and patients keep responsibilities clear and secure.
- **Dashboards and Analytics:** Overcrowding trends, doctor shift allocations, and patient visit logs help hospital superintendents make informed capacity and staffing decisions.
- **Modular and Scalable Design:** Separate modules and a document database support growth in hospitals, doctors, and patient visits.
- **Responsive Interface:** The web interface works across desktop, tablet, and mobile browsers.
- **Simple and Low-Cost:** The system is a practical and low-cost MERN-based solution that helps government hospitals balance patient queues efficiently.

7.2 Limitations

- **Internet Dependency:** An active network connection is required for access to the web application.
- **Rule-Based Triage:** The triage evaluates the symptom severity submitted by the patient and is not a replacement for the clinical diagnosis of a doctor.
- **Limited Evaluation:** The application has been demonstrated with a small number of hospitals, doctors, and patients and has not been evaluated in a large real-world deployment.
- **Verification by Sub-Admins:** The credential proofs of temporary doctors must be verified by the hospital sub-administrators before shifts can be assigned.
- **Basic Notification Mechanism:** Notifications are provided within the application and do not use advanced external notification services.
- **Security Enhancements Needed:** Although JWT authentication and role-based permissions are used, stronger measures such as audit logging and compliance with healthcare data regulations would be needed for a production deployment.

7.3 Screenshots

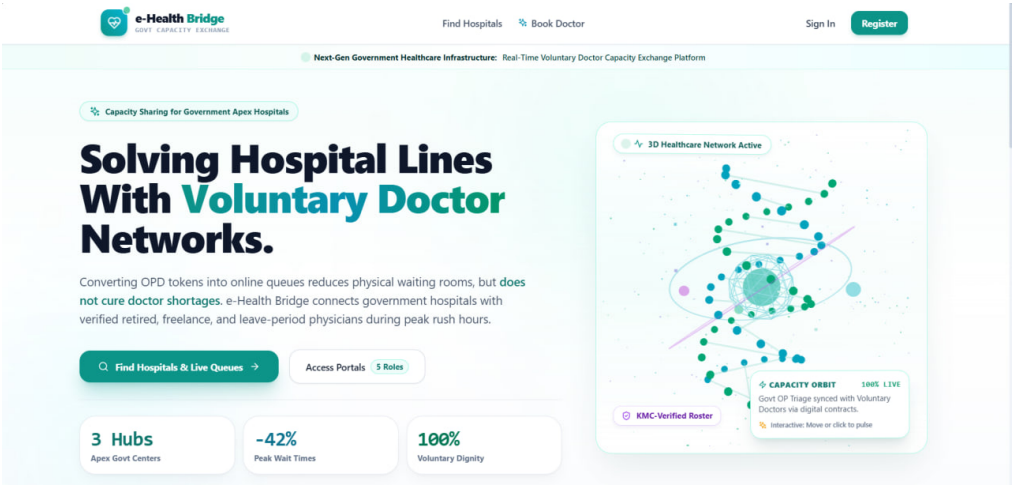


Figure 7.1: Home Page

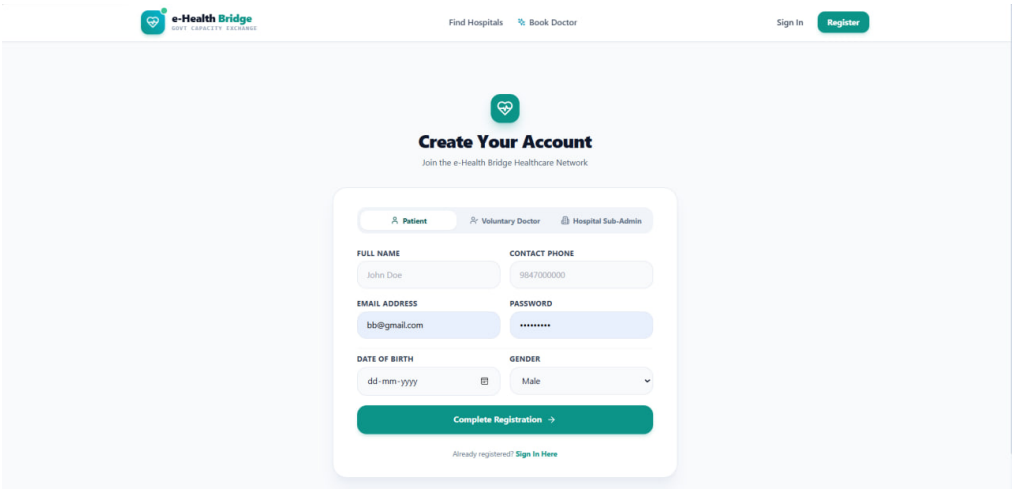


Figure 7.2: User Registration

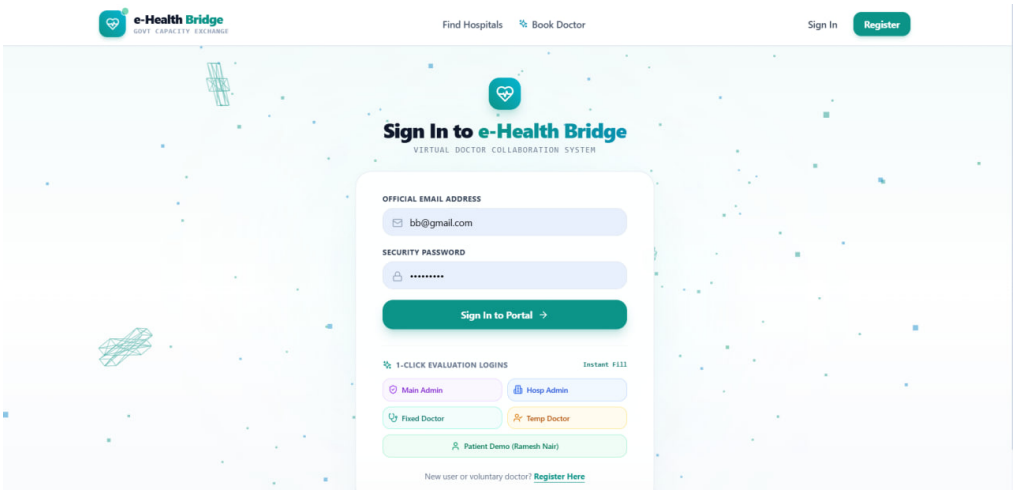


Figure 7.3: User Login

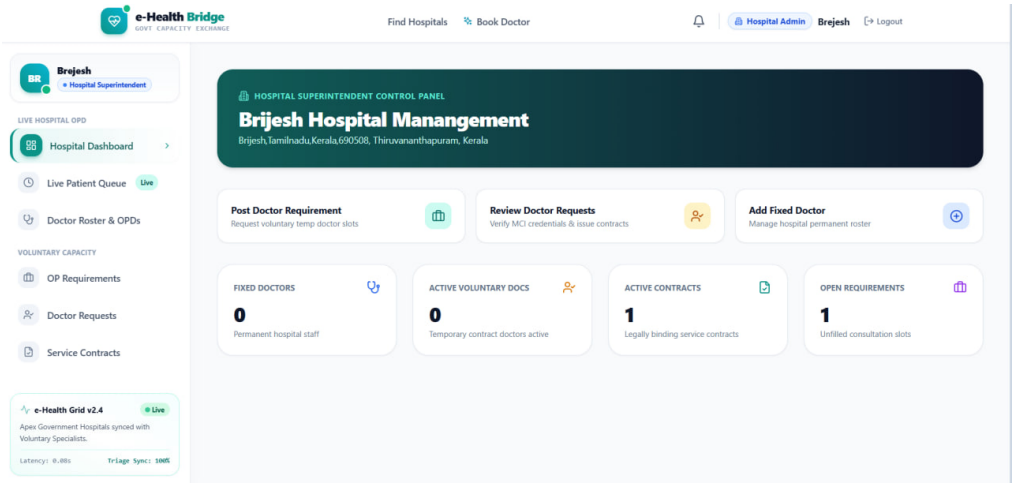


Figure 7.4: Hospital Admin Dashboard

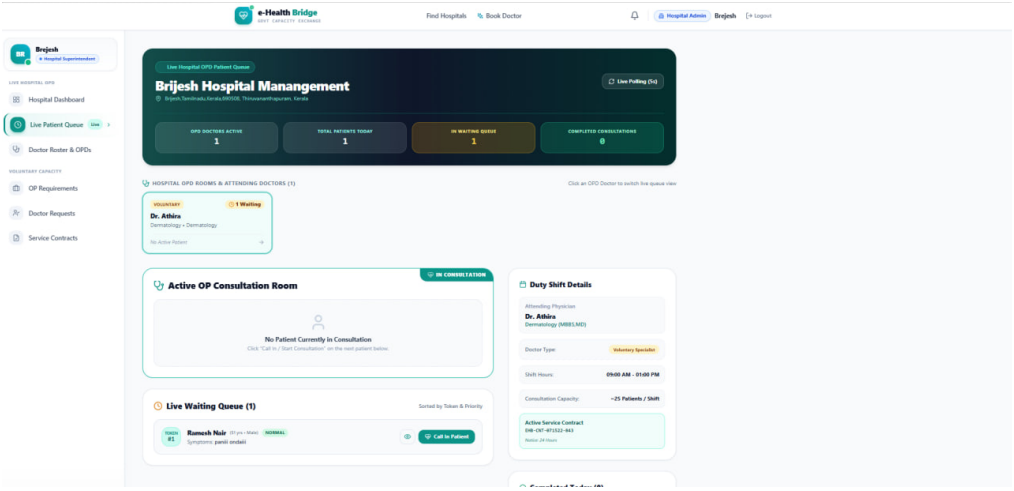


Figure 7.5: Live Patient Queue

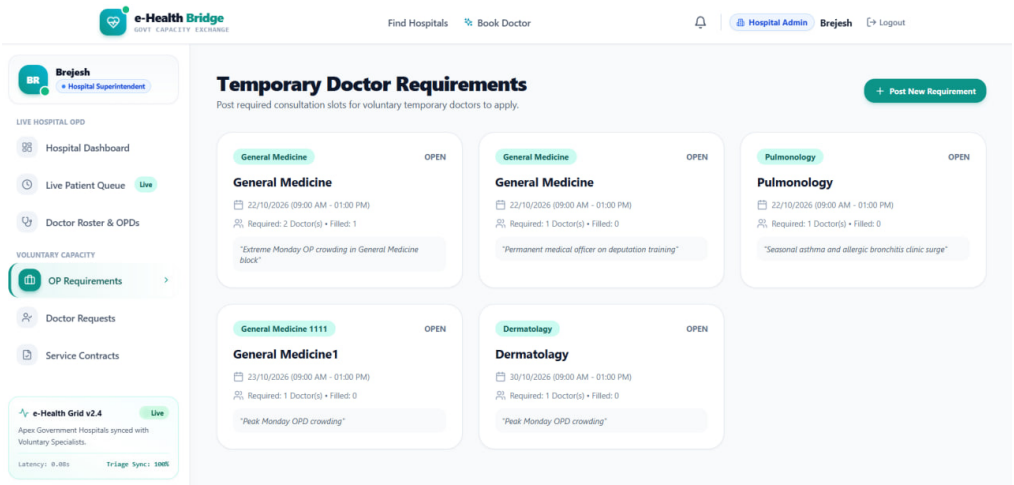


Figure 7.6: Temporary Doctor Requirements

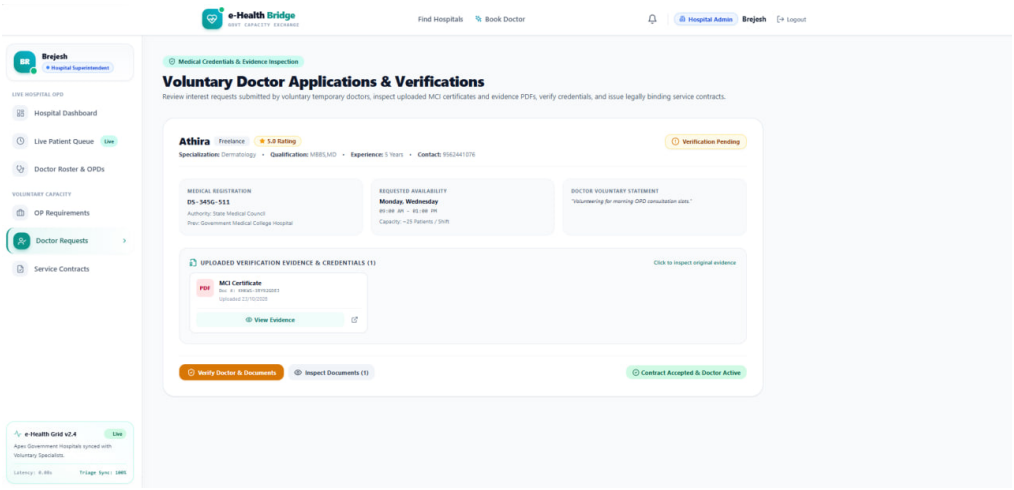


Figure 7.7: Voluntary Doctor Applications and Verifications

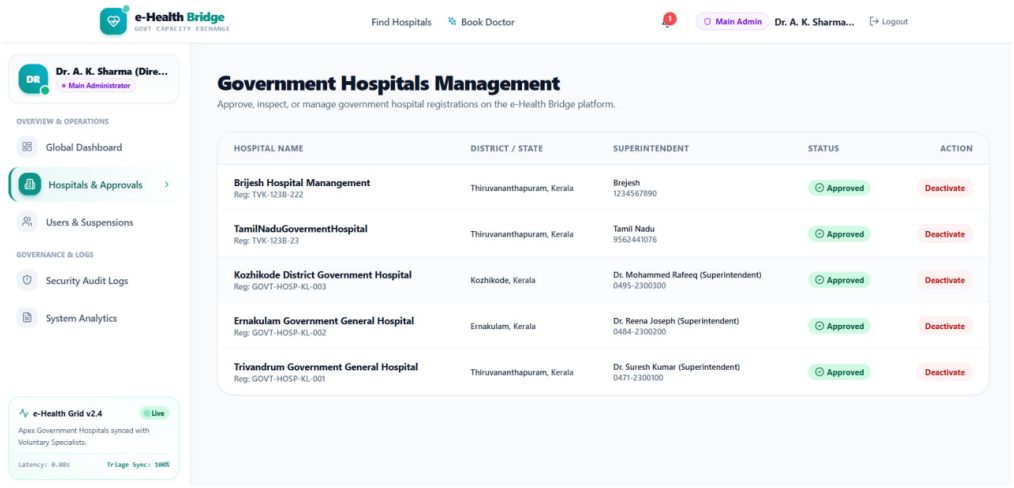


Figure 7.8: Government Hospitals Management

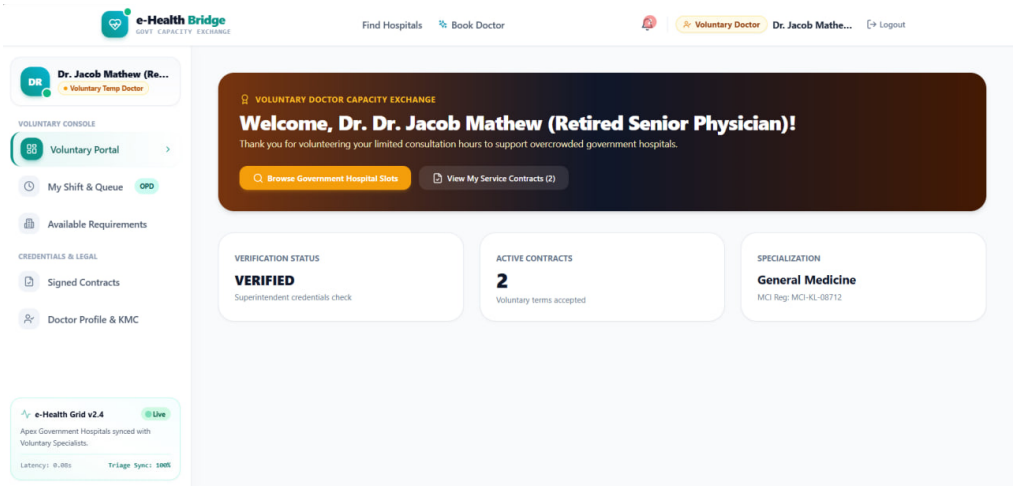


Figure 7.9: Voluntary Doctor Portal

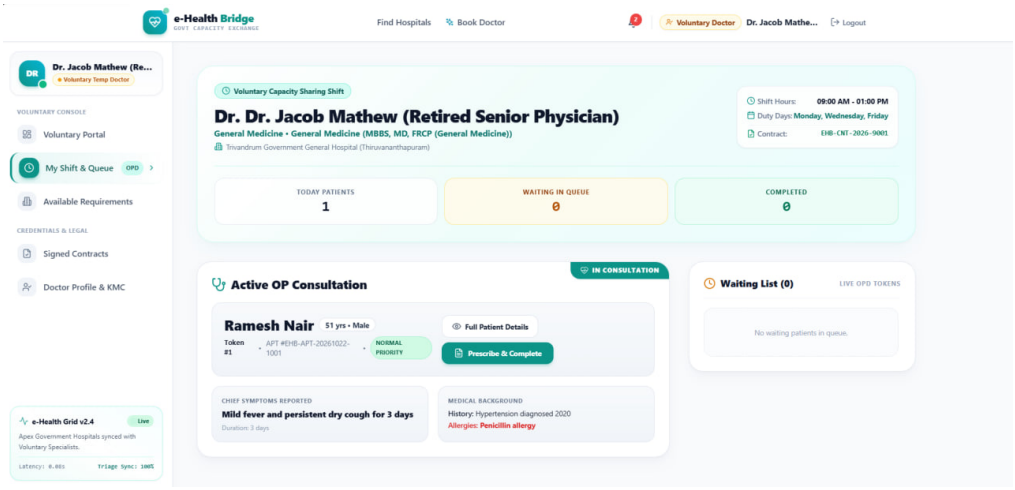


Figure 7.10: Doctor Shift and Active Consultation

CHAPTER 8

CONCLUSION AND FUTURE SCOPE

8.1 Conclusion

The development of the Virtual-Doctor Collaboration System addresses a practical problem of government hospitals, namely the heavy OPD crowd that cannot be reduced by online ticket booking alone. By connecting freelance and off-duty physicians to temporary shifts using digital contracts, the system expands the clinical capacity of a hospital dynamically, and by categorizing the symptom severity of patients, it optimizes the queue distribution and shortens the waiting delays.

Role-based workflows separate the responsibilities of the main administrator, hospital sub-administrators, doctors, and patients. The current MERN-based implementation facilitates streamlined doctor roster management and improved healthcare delivery, and it assists hospital superintendents in balancing heavy outpatient crowds efficiently.

8.2 Future Scope

1. **Advanced Triage:** Using machine learning on the collected symptom and visit data to improve the classification of symptom severity.
2. **Demand Prediction:** Using historical patient visit data to predict the crowd at different hospitals and times so that temporary doctors can be planned in advance.
3. **Telemedicine Integration:** Allowing temporary doctors to consult patients remotely through video consultation.
4. **Mobile Application and Notifications:** Developing a mobile application and adding SMS, email, or push notifications for appointments, contracts, and queue updates.
5. **Integration with Hospital Systems:** Integrating with existing government hospital records and registration systems so that patient data and prescriptions are shared automatically.

REFERENCES

[1] S. B. Saini, D. Singh, K. C. Sharma, and D. K. Saini, "AI-enhanced telemedicine: transforming resource allocation and cost-efficiency analysis via advanced queueing model," *Scientific Reports*, vol. 15, no. 1, Art. no. 31551, Aug. 2025, doi: 10.1038/s41598-025-15664-8.

[2] V. Mehta, P. Ajmera, S. Kalra, M. Miraj, R. Gallani, R. A. Shaik, H. A. Serhan, and R. Sah, "Human resource shortage in India's health sector: a scoping review of the current landscape," *BMC Public Health*, vol. 24, no. 1, Art. no. 1368, May 2024, doi: 10.1186/s12889-024-18850-x.

[3] Y.-C. Sun, O. Cosgun, and R. Sharman, "Optimizing telehealth utilization during COVID-19: enhancing efficiency and equity through data envelopment analysis and machine learning," *Operational Research*, vol. 25, no. 4, Art. no. 109, 2025, doi: 10.1007/s12351-025-00978-2.

[4] A. Riis, S. M. Erlandsen, and H. Andersson, "Integrated resource scheduling for patient appointment series—Designing tactical blueprints for an orthopedic clinic," *Operations Research and Decisions*, vol. 2025, Art. no. 200475, 2025, doi: 10.1016/j.ordal.2025.200475.